FPT UNIVERSITY

Requirement & Design Specification

Crowndine - Restaurant Management System(RMS)

Subject: SWD392

Version: 1.0

– Da Nang, July 2026 –

Record of Changes

Version	Date	In Charge	Change Description
1.0	July 2026	Project Team	Initial version

* A - Added M - Modified D - Deleted

Table of Contents

I Requirement Specification

I.1 Problem Description

Crowndine is a restaurant management system designed to support daily restaurant operations, including customer ordering, staff coordination, menu management, reservation handling, and order tracking. The system aims to reduce manual effort, improve service accuracy, and provide a centralized platform for restaurant workflows.

I.2 Major Features

The major features include online menu browsing, table reservation, order placement, order status monitoring, staff task management, menu administration, reporting, and role-based access control.

I.2.1 Features for customers are as follows

- Browse restaurant information, menu items, prices, and availability.
- Create reservations and manage booking information.
- Place dine-in or online orders and track order status.
- Review order history and submit feedback.

I.2.2 Features for staff are as follows

- Manage incoming orders and update order preparation status.
- Handle reservations, table assignments, and customer requests.
- Maintain menu items, categories, prices, and availability.
- Generate operational reports for management review.

I.3 System Context

The system interacts with customers, restaurant staff, administrators, payment services, notification services, and the restaurant database. External services may be integrated depending on implementation scope.

I.4 Non-Functional Requirements

Non-functional requirements describe the quality attributes and operating constraints that the Crowndine Restaurant Management System must satisfy. These requirements ensure that the system is not only functionally correct but also practical, secure, reliable, and maintainable in a real restaurant environment.

- **Usability:** The system shall provide an intuitive, seamless experience for all roles. For customers, the 4-step booking journey, interactive Table Map, and vibrant digital menu must be highly accessible. For staff and kitchen personnel, the Calendar View and KDS interfaces must be visually clear to manage real-time operations without requiring extensive technical training.
- **Reliability:** CrownDine must guarantee absolute accuracy in preserving reservations, order batching sequences, and financial records processed via PayOS (QR Code and Bank Transfer). The system must consistently maintain state synchronization across all modules and reliably execute the automated cancellation and 80% refund policy without any data loss or manual intervention.
- **Performance:** The platform shall process core actions—such as interactive table selection, cart updates, and payment authorizations—with minimal latency. The Kitchen Display System (KDS) must efficiently utilize WebSockets to ensure instant, real-time status updates under heavy loads, while the integrated Gemini AI chatbot must deliver prompt analytical insights.
- **Security:** All user data and transactions shall be strictly protected. The system must implement robust authentication utilizing JWT and Google OAuth2. Furthermore, a granular role-based access control (RBAC) mechanism must precisely isolate permissions among Admin, Staff, Kitchen, and Customer roles to prevent unauthorized system modifications.
- **Maintainability:** The software architecture shall be highly modular, enabling developers to seamlessly implement updates to the WebSockets infrastructure, PayOS integrations, and AI features without system downtime. This ensures easy future expansion and allows administrators to flexibly configure business rules via the admin panel rather than deep code changes.

I.5 Functional Requirements

FR1. The system shall allow customers to view menu items and restaurant information.

FR2. The system shall allow customers to create and manage reservations.

FR3. The system shall allow customers to place and track orders.

FR4. The system shall allow staff to confirm, update, and complete orders.

FR5. The system shall allow administrators to manage users, menu items, and reports.

I.5.1 Use Case Diagrams

This section should be completed by the project team with project-specific details, diagrams, tables, and references for the Crowndine Restaurant Management System.

I.5.2 Use Case Descriptions

I.5.3 Activity Diagrams

This section should be completed by the project team with project-specific details, diagrams, tables, and references for the Crowndine Restaurant Management System.

I.6 Data Requirements

Core data entities include User, Customer, Staff, Menu Item, Category, Reservation, Table, Order, Order Detail, Payment, Feedback, and Notification.

II Analysis Models

II.1 Interaction Diagrams

This section should be completed by the project team with project-specific details, diagrams, tables, and references for the Crowndine Restaurant Management System.

II.2 State Diagram

The main order lifecycle may include Created, Confirmed, Preparing, Ready, Served, Completed, and Cancelled states. State transitions must be triggered only by authorized users or valid system events.

III Design Specification

III.1 Integrated Communication Diagrams

This section should be completed by the project team with project-specific details, diagrams, tables, and references for the Crowndine Restaurant Management System.

III.2 System High-Level Design

The system follows a layered architecture consisting of presentation, application service, domain logic, data access, and persistence layers. Each layer separates responsibilities to improve maintainability and testability.

III.3 Component & Package Diagram

This section should be completed by the project team with project-specific details, diagrams, tables, and references for the Crowndine Restaurant Management System.

III.4 Detailed Design

Detailed design shall define classes, interfaces, service contracts, validation rules, exception handling, and data transfer objects for each major module.

III.5 Database Design

The database design shall include an entity relationship diagram, table schemas, primary keys, foreign keys, constraints, indexes, and sample records where applicable.

IV Implementation

IV.1 Project Architecture

The implementation should organize modules by responsibility, such as user management, menu management, reservation management, order management, payment integration, notification, and reporting.

IV.2 Mapping Class Diagrams & Interaction Diagrams to Code

Class diagrams shall be mapped to source code classes, interfaces, models, repositories, controllers, and services. Interaction diagrams shall be mapped to method calls, API flows, and transaction boundaries.

IV.3 Applying Alternative Architecture Patterns

Alternative patterns such as Model–View–Controller, layered architecture, repository pattern, and dependency injection may be applied where appropriate to improve separation of concerns.

V Service-Oriented Architecture (SOA)

V.1 Service Discovery Pattern

The Service Discovery Pattern enables services to locate each other dynamically through a service registry. In the Crowndine system, this pattern may be applied if the project evolves into independent services such as order service, menu service, payment service, and notification service.